

DD: SUB-WF-RESTRICT-01-FLOW-WIK — KE WIK Restriction BPMN

Developer implementation guide: DD_SUB-WF-RESTRICT-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow DD_SUB-WF-RESTRICT-01-FLOW-WIK-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Satellite of **DD_SUB-WF-RESTRICT-01-Subscription_Restriction** (parent). Defines the KE WIK reference BPMN for ADD + REMOVE restriction operations. BPMN process key: sub-restrict-wik. Composes 8 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. Both intents (ADD and REMOVE) share the same BPMN; intent is carried as a process variable.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-RESTRICT-01-FLOW-WIK-v1.0.md
Version	v1.0
DD type	operator BPMN flow
BPMN/process keys	sub-restrict-wik
Drools packages	rules.subscription.restrict.wik

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/subscriptions/sub_01HZ_KAMAU_FIBER/restrictions/OUTGOING_VOICE_BARRED
- POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/restrictions

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- drools-decide
- ful-call-restrict
- fulfillment-completed
- fulfillment-failed
- kafka-emit
- manual-recovery-decision
- notification-send
- put-active-restrictions
- restrict-remove
- sub-lm-compute-cycle-window
- sub-lm-put-active-restrictions
- sub-restrict-wik
- subscription-operation-finalize
- subscription-workflow-rules
- validate-dunning-marker
- validate-preconditions
- validate-role-gating

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionRestrictionAdded
- SubscriptionRestrictionRemoved

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.

4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Satellite of **DD_SUB-WF-RESTRICT-01-Subscription_Restriction** (parent). Defines the KE WIK reference BPMN for ADD + REMOVE restriction operations. BPMN process key: sub-restrict-wik. Composes 8 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. Both intents (ADD and REMOVE) share the same BPMN; intent is carried as a process variable.

1. What this flow is

KE WIK's BPMN composes the framework workers for restriction operations. Trigger endpoint accepts the request (from BIL-04 dunning, BO admin, call center, customer portal), persists subscription_operation row, starts process instance of sub-restrict-wik with intent variable set to ADD or REMOVE. The BPMN orchestrates: Drools validation → billing intent → FUL-04 call (APPLY or LIFT depending on intent) → put-active-restrictions (append for ADD, remove for REMOVE) → emit domain event → notification → finalize operation.

Workers composed (8)

#	Worker topic	Used for
1	drools-decide	Validation: subscription ACTIVE, restriction catalog lookup, role-trigger gating, dunning-marker checks (REMOVE only), customer-self-service-eligibility
2	sub-lm-compute-cycle-window	Compute cycle window for billing intent context
3	bil01-emit-intent	RESTRICTION_ADD_INTENT or RESTRICTION_REMOVE_INTENT — billing decides any applicable fees (KE v1.0: no fees)
4	ful-call-restrict	FUL-04 — APPLY_RESTRICTION (ADD) or LIFT_RESTRICTION (REMOVE)
5	sub-lm-put-active-restrictions	Mutate active_restrictions[] array on subscription

#	Worker topic	Used for
6	kafka-emit	Emit SubscriptionRestrictionAdded or SubscriptionRestrictionRemoved event via outbox
7	notification-send	NOT-01 customer notification (reliability-isolated)
8	subscription-operation-finalize	Set subscription_operation.final_state = COMPLETED

Message catches

Type	Name	Used when
Message catch	fulfillment-completed / fulfillment-failed (correlated on fulfillmentCorrelationId)	After ful-call-restrict; BPMN waits for FUL-04

Drools package

rules.subscription.restrict.wik — declared decisions:

Decision	Output	Used at
validate-preconditions	eligible: bool, eligibilityReasonCode?, restrictionCatalogSnapshot: object	Beginning — subscription ACTIVE, restriction code exists + is_active, intent-specific checks (already restricted for ADD; not applied for REMOVE)
validate-role-gating	roleAllowed: bool, roleReasonCode?	Role match per trigger, customer-self-service eligibility, admin-only enforcement
validate-dunning-marker	removeAllowed: bool, removeReasonCode?	REMOVE only — checks dunning marker on existing restriction entry; rejects human-actor removal unless override allowed

2. BPMN structure

Process key: sub-restrict-wik. Deployed as sub-restrict-wik.bpmn in the Camunda engine.

2.1 Outline

```

START (event)
  ■ Carries process variables from trigger endpoint:
  ■ operationId, subscriptionId, customerId, operatorCode, packageRef, homepassId,
  ■ intent ("ADD" or "REMOVE"), restrictionCode, activationTrigger, notes,
  ■ dunningReference (if DUNNING_DRIVEN), dunningOverride (REMOVE only),
  ■ initiatingActorUserId, initiatingActorRole, initiatingWorkflowKind,
  ■ initiatingWorkflowRef, idempotencyKey
  ▼
[STATE: VALIDATING]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.restrict.wik",
         factsJson={subscription state, active_restrictions[], restriction catalog row,
                    intent, activationTrigger, role, dunningOverride,
                    requestedDecisions=["validate-preconditions", "validate-role-gating",
                                        "validate-dunning-marker" (if intent==REMOVE)]}
  outputs: eligible, eligibilityReasonCode, restrictionCatalogSnapshot,
          roleAllowed, roleReasonCode, removeAllowed?, removeReasonCode?
  ■
  ▼ XOR: all checks pass?
  ■ NO → REJECT subflow (emit *Rejected event + finalize FAILED)
  ■ YES ↓
  ■
Service Task: sub-lm-compute-cycle-window
  inputs: subscriptionId, referenceDate=NOW
  outputs: cycleWindowJson
  ▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
         intentKind=<"RESTRICTION_ADD_INTENT" if intent==ADD else "RESTRICTION_REMOVE_INTENT">,
         intentContext={restrictionCode, fulfillmentAction:<from catalog snapshot>,
                       activationTrigger, dunningMarker:<true if DUNNING_DRIVEN>,
                       dunningReference, currentActiveRestrictions},

```

```

        cycleWindow
    outputs: intentDecisionId, intentDecision (lineItems, totals; usually empty for KE v1.0)
    ■
    ▼
[STATE: FULFILLMENT_CALL]
Service Task: ful-call-restrict
    inputs: subscriptionId,
            intent=<"APPLY_RESTRICTION" if intent==ADD else "LIFT_RESTRICTION">,
            restrictionCode, fulfillmentAction:<from catalog snapshot>,
            reasonCategory:<activationTrigger>,
            remainingActiveRestrictions:<for REMOVE only – array of codes still active after this removal>,
            fulfillmentCorrelationId=<operationId>:restrict
    outputs: restrictionFulfillmentId, restrictionTerminalState
    ■
    ▼ [STATE: AWAITING_FULFILLMENT_RESPONSE]
Message Catch: fulfillment-completed (correlated on fulfillmentCorrelationId)
    OR
Message Catch: fulfillment-failed
    ■ FAILED → REVERT subflow (no state-revert needed – array hasn't been mutated yet – just emit *Rejected + finalize FAILURE)
    ■ COMPLETED ↓
    ■
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-active-restrictions
    inputs: subscriptionId,
            restrictionsOperationKind=<"ADD" or "REMOVE">,
            restrictionCode,
            restrictionMetadata=<for ADD: {addedAt, addedByActor, addNotes, dunningMarker, dunningReference};
                                for REMOVE: {removedAt, removedByActor, removeNotes, dunningOverride}>
    outputs: lmEventId, newActiveRestrictions
    ■
    ▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
    inputs: topic=<"sophix.subscription.subscription.restrictionadded" or
                "sophix.subscription.subscription.restrictionremoved">,
            key=<subscriptionId>,
            payloadJson=<full event per parent $6>
    ■
Service Task: notification-send (reliability-isolated)
    inputs: recipientKind="customer", recipientRef=<customerId>,
            notificationEventKind=<"SUBSCRIPTION_RESTRICTION_ADDED" or "SUBSCRIPTION_RESTRICTION_REMOVED">,
            eventContext={restrictionCode, dunningMarker, dunningReference?,
                            outstandingDebtAmount?, outstandingDebtCurrency?, restrictionAddedAt?, restrictionRemovedAt?}
    ■ failure here does NOT fail the BPMN
    ▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
    inputs: operationId, finalState="COMPLETED"
    ■
END (event)

```

2.2 BPMN XML excerpt — intent-conditional fulfillment + array mutation

```

<bpmn:serviceTask id="Task_ful_restrict" name="Call FUL-04"
    camunda:type="external" camunda:topic="ful-call-restrict">
    <bpmn:extensionElements>
        <camunda:inputOutput>
            <camunda:inputParameter name="intent">${intent == "ADD" ? "APPLY_RESTRICTION" : "LIFT_RESTRICTION"}</camunda:inputParameter>
        </camunda:inputOutput>
    </bpmn:extensionElements>
</bpmn:serviceTask>

<bpmn:serviceTask id="Task_put_active_restrictions" name="Update active_restrictions[]"
    camunda:type="external" camunda:topic="sub-lm-put-active-restrictions">
    <bpmn:extensionElements>
        <camunda:inputOutput>
            <camunda:inputParameter name="restrictionsOperationKind">${intent}</camunda:inputParameter>
        </camunda:inputOutput>
    </bpmn:extensionElements>
</bpmn:serviceTask>

<bpmn:intermediateCatchEvent id="Catch_fulfillment_done">
    <bpmn:messageEventDefinition messageRef="Message_fulfillment_completed" />
</bpmn:intermediateCatchEvent>

<bpmn:intermediateCatchEvent id="Catch_fulfillment_failed">
    <bpmn:messageEventDefinition messageRef="Message_fulfillment_failed" />
</bpmn:intermediateCatchEvent>

```

2.3 Compensation paths

Restrictions are a simpler compensation case than ACTIVATE/TERMINATE because the subscription's status_code never changes. The only mutation that needs reverting on partial failure is the active_restrictions[] array — but since the BPMN orders fulfillment BEFORE the array mutation (FUL-04 must succeed before SUB-LM-01 commits), there's no partial-commit window to compensate.

If this step fails	What happens
drools-decide validation	No mutations yet; emit *Rejected event; finalize FAILED
bil01-emit-intent	No state mutations; emit *Rejected; finalize FAILED
ful-call-restrict returns FAILED	No state mutations; emit *Rejected; finalize FAILED. Note: FUL-04 internally handles network-side compensation per its own SCO rules.
sub-lm-put-active-restrictions	This SHOULD NOT fail (it's idempotent + atomic at the DB level). If it does (DB unavailable), the network-side restriction has been applied but the array isn't updated → manual recovery via manual-recovery-decision user task.
kafka-emit	Retried by outbox dispatcher; BPMN proceeds
notification-send	Logged but NOT compensated (reliability-isolated); BPMN proceeds

2.4 Configuration row (KE WIK)

In subscription_operation_config (framework parent table):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
WIK	RESTRICT	sub-restrict-wik	60	{"customer_self_service_enabled": true}

In subscription_restrict_config (parent §3.1):

operator_code	customer_self_service_enabled	dunning_marker_strict	restriction_approval_required_for_codes
WIK	true	true	[]

3. Drools package rules.subscription.restrict.wik

KE WIK rules deployed as part of subscription-workflow-rules KJAR.

3.1 validate-preconditions

```
package rules.subscription.restrict.wik

import com.sophix.subwf.facts.RestrictRequestFact
import com.sophix.subwf.facts.SubscriptionStateFact
import com.sophix.subwf.facts.RestrictionCatalogFact

rule "Eligibility: subscription must be ACTIVE"
when
    $req: RestrictRequestFact()
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
then
    modify($req) { setEligible(false); setEligibilityReasonCode("INVALID_STATE_FOR_RESTRICTION") }
end

rule "Eligibility: restriction code must exist + be active"
when
    $req: RestrictRequestFact(eligible == null)
    not (RestrictionCatalogFact(code == $req.restrictionCode, isActive == true))
then
    modify($req) { setEligible(false); setEligibilityReasonCode("UNKNOWN_OR_INACTIVE_RESTRICTION_CODE") }
end

rule "Eligibility ADD: not already restricted"
when
    $req: RestrictRequestFact(intent == "ADD", eligible == null)
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId)
    eval($sub.getActiveRestrictionCodes().contains($req.getRestrictionCode()))
then
    modify($req) { setEligible(false); setEligibilityReasonCode("ALREADY_RESTRICTED") }
end

rule "Eligibility REMOVE: must be applied"
when
```

```

    $req: RestrictRequestFact(intent == "REMOVE", eligible == null)
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId)
    eval(!$sub.getActiveRestrictionCodes().contains($req.getRestrictionCode()))
then
    modify($req) { setEligible(false); setEligibilityReasonCode("RESTRICTION_NOT_APPLIED") }
end

rule "Eligibility: default-pass"
salience -100
when
    $req: RestrictRequestFact(eligible == null)
then
    modify($req) { setEligible(true) }
end

```

3.2 validate-role-gating

```

rule "Role: BACKOFFICE_OPERATOR requires SUBSCRIPTION_OPERATOR or SUBSCRIPTION_ADMIN"
when
    $req: RestrictRequestFact(activationTrigger == "BACKOFFICE_OPERATOR",
                             initiatingActorRole not in ("SUBSCRIPTION_OPERATOR", "SUBSCRIPTION_ADMIN"),
                             roleAllowed == null)
then
    modify($req) { setRoleAllowed(false); setRoleReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Role: CUSTOMER_SELF_SERVICE requires operator config + catalog eligibility"
when
    $req: RestrictRequestFact(activationTrigger == "CUSTOMER_SELF_SERVICE", roleAllowed == null)
    $cat: RestrictionCatalogFact(code == $req.restrictionCode)
    $cfg: OperatorConfigFact()
    eval(!$cfg.isCustomerSelfServiceEnabled() || !$cat.isCustomerSelfServiceEligible())
then
    modify($req) { setRoleAllowed(false); setRoleReasonCode("CUSTOMER_SELF_SERVICE_NOT_ALLOWED") }
end

rule "Role: admin_only catalog row requires SUBSCRIPTION_ADMIN"
when
    $req: RestrictRequestFact(roleAllowed == null)
    $cat: RestrictionCatalogFact(code == $req.restrictionCode, adminOnly == true)
    $req: RestrictRequestFact(initiatingActorRole != "SUBSCRIPTION_ADMIN")
then
    modify($req) { setRoleAllowed(false); setRoleReasonCode("ADMIN_ONLY_RESTRICTION") }
end

# ... etc for DUNNING_DRIVEN, CALL_CENTER, default-pass

```

3.3 validate-dunning-marker

```

rule "Dunning: REMOVE on dunning-marked restriction by BILLING_INTERNAL always allowed"
when
    $req: RestrictRequestFact(intent == "REMOVE", initiatingActorRole == "BILLING_INTERNAL")
then
    modify($req) { setRemoveAllowed(true) }
end

rule "Dunning: REMOVE on dunning-marked by human actor when strict – block"
when
    $req: RestrictRequestFact(intent == "REMOVE", removeAllowed == null,
                             initiatingActorRole not in ("BILLING_INTERNAL"))
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId)
    $cfg: OperatorConfigFact(dunningMarkerStrict == true)
    eval($sub.findActiveRestriction($req.getRestrictionCode()).isDunningMarker())
then
    modify($req) {
        setRemoveAllowed(false);
        setRemoveReasonCode("SYSTEM_MANAGED_RESTRICTION")
    }
end

rule "Dunning: REMOVE on dunning-marked by SUBSCRIPTION_ADMIN with override when not strict – allowed"
when
    $req: RestrictRequestFact(intent == "REMOVE", removeAllowed == null,
                             initiatingActorRole == "SUBSCRIPTION_ADMIN",
                             dunningOverride == true)
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId)
    $cfg: OperatorConfigFact(dunningMarkerStrict == false)
    eval($sub.findActiveRestriction($req.getRestrictionCode()).isDunningMarker())
then
    modify($req) { setRemoveAllowed(true) }
end

```

```

rule "Dunning: default-pass for non-dunning-marked restrictions"
salience -100
when
    $req: RestrictRequestFact(intent == "REMOVE", removeAllowed == null)
then
    modify($req) { setRemoveAllowed(true) }
end

```

4. Worked example — Kamau gets voice-barred at BIL-04 dunning L2

Kamau's invoice INV_01HZ went unpaid past the L2 dunning threshold. BIL-04 advances Kamau's dunning state to L2 and calls SUB-WF-RESTRICT-01 to add OUTGOING_VOICE_BARRED.

02:00:00 — Trigger endpoint:

```

POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/restrictions
Idempotency-Key: restrict-add-sub_01HZ_KAMAU-OUTGOING_VOICE_BARRED-dunning-L2-20260521

{ "operatorCode": "WIK", "restrictionCode": "OUTGOING_VOICE_BARRED",
  "activationTrigger": "DUNNING_DRIVEN",
  "dunningReference": "bil04-dunning-wave-2026-05-21-L2",
  "notes": "Dunning L2 – voice outgoing barred until invoice INV_01HZ paid",
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-dunning-wave-2026-05-21-L2" }
→ 202 operationId=subop_01HZ_KAMAU_RESTRICT_ADD, currentState=VALIDATING

```

subscription_operation row inserted. Camunda starts pi_01HZ_KAMAU_RESTRICT_ADD of sub-restrict-wik.

02:00:00.5 — VALIDATING: drools-decide invokes rules.subscription.restrict.wik with [validate-preconditions, validate-role-gating] (no dunning-marker check needed for ADD). Facts: subscription ACTIVE, restriction code OUTGOING_VOICE_BARRED exists + is_active=true, not currently in Kamau's active_restrictions[], role BILLING_INTERNAL matches DUNNING_DRIVEN trigger. Returns eligible=true, roleAllowed=true, restrictionCatalogSnapshot={code: "OUTGOING_VOICE_BARRED", fulfillment_action: "AAA_RESTRICT_OUTGOING_VOICE", customer_self_service_eligible: false, admin_only: false}.

02:00:01: sub-lm-compute-cycle-window returns {cycleStart:"2026-05-01", cycleEnd:"2026-05-31"}.

02:00:01.5 — BILLING CALL: bil01-emit-intent calls BIL-01 with intentKind=RESTRICTION_ADD_INTENT, context {restrictionCode: OUTGOING_VOICE_BARRED, fulfillmentAction: AAA_RESTRICT_OUTGOING_VOICE, activationTrigger: DUNNING_DRIVEN, dunningMarker: true, dunningReference: ...}. BIL-01 evaluates BillableEvents filtered on restriction code — KE v1.0 has none for this restriction. Returns lineItems: [], totals.net: 0.

02:00:02 — FULFILLMENT CALL: ful-call-restrict calls FUL-04 with intent=APPLY_RESTRICTION, restrictionCode=OUTGOING_VOICE_BARRED, fulfillmentAction=AAA_RESTRICT_OUTGOING_VOICE, reasonCategory=DUNNING_DRIVEN, fulfillmentCorrelationId=subop_01HZ_KAMAU_RESTRICT_ADD:restrict. FUL-04 dispatches to VoipSipProvisioner (push AAA rule to VoipSwitch via side channel to bar EXT-5421's outgoing calls) and GponResProvisioner (returns no-op for voice-restriction on data plane). Returns terminalState=FULLY_FULFILLED after 1.4 seconds.

02:00:03.4 — AWAITING_FULFILLMENT_RESPONSE: Camunda message catch fulfillment-completed fires.

02:00:04 — COMMITTING_FINAL_STATE: sub-lm-put-active-restrictions appends to Kamau's active_restrictions[]:

```

{
  "restrictionCode": "OUTGOING_VOICE_BARRED",
  "addedAt": "2026-05-21T02:00:04+03:00",
  "addedByActorUserId": "bil04-svc-account",
  "addedByActorRole": "BILLING_INTERNAL",
  "addNotes": "Dunning L2 – voice outgoing barred until invoice INV_01HZ paid",
  "dunningMarker": true,
  "dunningReference": "bil04-dunning-wave-2026-05-21-L2"
}

```

02:00:04.2 — EMITTING_EVENT: kafka-emit writes full SubscriptionRestrictionAdded payload to outbox. Outbox dispatcher publishes within seconds.

02:00:04.5: notification-send invokes NOT-01 with notificationEventKind=SUBSCRIPTION_RESTRICTION_ADDED and event context {restrictionCode: "OUTGOING_VOICE_BARRED", dunningMarker: true, dunningReference: "bil04-dunning-wave-2026-05-21-L2"}. NOT-01 resolves Kamau's preferred channel + template + language, then dispatches the restriction notification.

02:00:05 — COMPLETED: subscription-operation-finalize sets final_state=COMPLETED. Total operation duration: ~5 seconds.

02:00:05+ (downstream):

- BIL-04 consumes SubscriptionRestrictionAdded, acknowledges its L2 step, schedules L3 dunning check for next escalation window if invoice remains unpaid.
- CVM analytics records the restriction event for retention modeling.

Kamau can still receive calls and use data. His outgoing voice line is barred until he pays.

Later — invoice paid, restriction lifted

June 1, 10:00: Kamau pays INV_01HZ via M-Pesa. BIL-04's resume-after-payment flow calls:

```
DELETE /api/subscriptions/sub_01HZ_KAMAU_FIBER/restrictions/OUTGOING_VOICE_BARRED
Idempotency-Key: restrict-remove-sub_01HZ_KAMAU-OUTGOING_VOICE_BARRED-bil04-resume-20260601

{ "operatorCode": "WIK",
  "notes": "Customer settled invoice INV_01HZ on 2026-06-01",
  "dunningOverride": false,
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-resume-2026-06-01" }
→ 202 operationId=subop_01HZ_KAMAU_RESTRICT_REMOVE
```

Same BPMN runs with intent=REMOVE. Drools rules:

- validate-preconditions: subscription ACTIVE, restriction in active_restrictions[] → eligible=true
- validate-role-gating: BILLING_INTERNAL on DUNNING_DRIVEN trigger → allowed
- validate-dunning-marker: BILLING_INTERNAL always allowed on dunning-marked → removeAllowed=true

ful-call-restrict calls FUL-04 with intent=LIFT_RESTRICTION, remainingActiveRestrictions=[] (after this lift, Kamau has no other restrictions). FUL-04's VoipSipProvisioner clears the AAA rule.

sub-lm-put-active-restrictions removes the entry. Notification template restrict-remove → SMS "Your service is fully restored. Thank you for your payment."

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-RESTRICT-01 (parent)	This satellite implements parent's contract	Triggers, intents, role-gating, dunning-marker rules all defined in parent
SUB-WF-FRAMEWORK-01 (framework parent)	This satellite composes framework primitives	subscription_operation row written; BPMN started
SUB-WF-FRAMEWORK-01-WORKERS	This satellite references workers by topic	8 workers composed
SUB-LM-01	Workers read catalog + write active_restrictions[]	Restriction catalog read; put-active-restrictions for array mutation
BIL-01	bil01-emit-intent worker	RESTRICTION_ADD_INTENT / RESTRICTION_REMOVE_INTENT
FUL-04	ful-call-restrict worker	APPLY_RESTRICTION or LIFT_RESTRICTION
BIL-04 Dunning	Calls trigger endpoints + consumes events	Dunning ladder L2 step
NOT-01	notification-send worker	Customer notification

5.1 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.

B — Current TZ

Status: To be filled in after codebase cartography review.

KE legacy uses an AAA restrict/restore pattern at dunning suspension; whether a structured restriction taxonomy exists or AAA rules are ad-hoc requires source-code review. Until that cartography lands, this satellite does not assert specific legacy structural claims.